

Laboratorio de Integración I

Del problema al programa

Propósito de esta clase: detenernos un momento, integrar lo aprendido y fortalecer el puente entre **comprender un problema** y **convertirlo en un programa**.

Este laboratorio no pretende volver a impartir los temas anteriores. El objetivo es diferente: resolver problemas de forma ordenada, comenzando por el análisis y aumentando gradualmente la dificultad hasta integrar variables, bifurcaciones, ciclos, menús, **for**, procedimientos, funciones y parámetros.

! Regla principal del laboratorio

No comenzamos escribiendo código.

Antes de abrir el compilador debemos ser capaces de explicar con nuestras propias palabras:

1. ¿Qué nos están pidiendo?
2. ¿Qué información conocemos o debemos solicitar?
3. ¿Qué resultados necesitamos obtener?
4. ¿Qué operaciones o decisiones debemos realizar?
5. ¿Qué estructura de programación podría ayudarnos?
6. ¿Cómo comprobaremos que nuestra solución funciona?

1. La ruta que seguiremos

En todos los ejercicios utilizaremos la misma ruta de razonamiento:

REQUERIMIENTO → **ANÁLISIS** → **SOLUCIÓN** → **PSEUDOCÓDIGO** → **C++**
→ **PRUEBA**

No importa si el problema es sencillo o complejo. La metodología será siempre la misma.

1.1 Antes de programar

Para cada problema responderemos primero estas preguntas:

Pregunta	¿Qué buscamos?
¿Cuál es el requerimiento?	Explicar qué debe resolver el programa.
¿Qué sabemos?	Datos disponibles y reglas del problema.
¿Qué necesitamos?	Entradas que deberá proporcionar el usuario.
¿Qué debemos obtener?	Salidas o resultados esperados.
¿Cómo podemos resolverlo?	Operaciones, decisiones y repeticiones necesarias.
¿Qué estructuras necesitamos?	Variables, if , switch , while , for , funciones, etc.
¿Cómo lo comprobamos?	Casos de prueba.

Consejo del Ingeniero

Si todavía no puede explicar la solución sin utilizar C++, **todavía no es momento de programar**.

Primero resolvemos el problema nosotros. Después le enseñamos a la computadora cómo hacerlo.

Ejercicio 1. Empecemos por lo esencial: variables y operaciones

El requerimiento

Una tienda necesita un programa que calcule el total que debe pagar un cliente por la compra de un producto.

El usuario debe ingresar:

- nombre del producto;
- precio unitario;
- cantidad comprada.

El programa debe mostrar:

- nombre del producto;
- subtotal de la compra.

Antes de programar

Respondamos juntos:

1. ¿Qué problema estamos resolviendo?
2. ¿Qué datos necesitamos solicitar?
3. ¿Cuáles de esos datos son texto, enteros o reales?
4. ¿Qué dato debemos calcular?
5. ¿Cuál es la fórmula?
6. ¿Qué información debe mostrarse al final?

Análisis

Entradas

- producto
- precio
- cantidad

Proceso

$$subtotal = precio \times cantidad$$

Salidas

- producto;
- subtotal.

Pensemos en las variables

Variable	Tipo sugerido	Función
producto	string	Guardar el nombre del producto
precio	double	Guardar el precio unitario
cantidad	int	Guardar la cantidad comprada
subtotal	double	Guardar el resultado

Pseudocódigo

```
Inicio
    Definir producto
    Definir precio, subtotal
    Definir cantidad

    Leer producto
    Leer precio
    Leer cantidad

    subtotal = precio * cantidad

    Mostrar producto
    Mostrar subtotal
Fin
```

Ahora sí: C++

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string producto;
    double precio, subtotal;
    int cantidad;

    cout << "Ingrese el nombre del producto: ";
    cin >> producto;

    cout << "Ingrese el precio: Q";
    cin >> precio;

    cout << "Ingrese la cantidad: ";
    cin >> cantidad;

    subtotal = precio * cantidad;

    cout << "\nProducto: " << producto << endl;
    cout << "Subtotal: Q" << subtotal << endl;
```

```
return 0;  
}
```

Caso de prueba

Producto	Precio	Cantidad	Resultado esperado
Cuaderno	Q15.00	4	Q60.00

i ¿Qué recuperamos aquí?

Variables, tipos de datos, entrada, proceso, salida, operadores aritméticos y asignación.

Ejercicio 2. Ahora debemos tomar una decisión

El requerimiento

La tienda decide aplicar un **10 % de descuento** cuando el subtotal de la compra sea mayor a Q500.

El programa debe mostrar:

- subtotal;
- descuento aplicado;
- total a pagar.

Analicemos antes de escribir código

La primera parte ya la conocemos:

$$subtotal = precio \times cantidad$$

Ahora aparece una pregunta:

¿El subtotal es mayor a Q500?

Si la respuesta es verdadera:

$$subtotal = precio \times cantidad$$

Si es falsa:

[descuento = 0]

Finalmente:

[total = subtotal - descuento]

¿Qué cambió respecto al ejercicio anterior?

Ya no basta con ejecutar instrucciones en secuencia. El programa debe **tomar una decisión**.

```
¿subtotal > 500?  
  |  
+---+---+  
|       |  
Sí       No  
|       |  
10 %    0 %
```

Antes de continuar

Complete:

Entradas:

Procesos:

Condición:

Salidas:

Pseudocódigo esperado

Construya primero el pseudocódigo utilizando una estructura **Si ... Sino**.

Reto de programación

Implemente la solución en C++ utilizando:

```
if (...) {  
    ...  
} else {  
    ...  
}
```

Casos de prueba obligatorios

Subtotal	¿Descuento?	Resultado esperado
Q300	No	Q300
Q500	No	Q500
Q600	Sí	Q540

 Observe el límite

El requerimiento dice **mayor a Q500**, no “mayor o igual”.
Por eso Q500 **no** recibe descuento.

Ejercicio 3. Una decisión puede tener varios caminos

El requerimiento

Crear un programa que solicite una nota entre 0 y 100 y muestre la clasificación correspondiente:

Nota	Clasificación
90–100	Excelente
80–89	Muy bueno
70–79	Bueno
61–69	Aprobado
0–60	Reprobado

Si la nota es menor que 0 o mayor que 100, debe mostrar:

Nota inválida

No programe todavía

Primero responda:

1. ¿Cuántas entradas existen?
2. ¿Cuántas posibles salidas existen?
3. ¿Qué debemos verificar antes de clasificar la nota?
4. ¿Importa el orden en que evaluemos las condiciones?
5. ¿Necesitamos un `if` simple o varias decisiones relacionadas?

Construya la lógica

Complete el siguiente esquema:

```
Si la nota es inválida
    -----
Sino si -----
    Mostrar "Excelente"
Sino si -----
    Mostrar "Muy bueno"
Sino si -----
    Mostrar "Bueno"
Sino si -----
    Mostrar "Aprobado"
Sino
    -----
```

Pruebas mínimas

Pruebe al menos:

-5
0
60
61
75
89
90
100
105

Pregunta clave

¿Por qué probamos precisamente valores cercanos a los límites?
Porque muchos errores de lógica aparecen en los extremos de los rangos.

Ejercicio 4. Cuando una tarea debe repetirse

El requerimiento

Un docente necesita ingresar las notas de **N estudiantes** y calcular el promedio general del grupo.

El programa debe:

1. preguntar cuántos estudiantes serán ingresados;
2. solicitar cada nota;
3. acumular las notas;
4. calcular el promedio;
5. mostrar el resultado.

Analicemos el problema

Hasta ahora podíamos escribir:

```
Leer nota1
Leer nota2
Leer nota3
```

Pero no sabemos anticipadamente cuántos estudiantes existirán.

Entonces aparece una nueva necesidad:

Repetir una misma acción N veces.

Preguntas de análisis

- ¿Qué dato controla cuántas veces repetimos?
- ¿Qué variable cambia en cada repetición?
- ¿Qué variable acumula las notas?
- ¿En qué momento calculamos el promedio?
- ¿Debemos utilizar `if`?
- ¿Debemos utilizar un ciclo?

Simulación manual

Supongamos:

```
N = 4

Notas:
70
80
90
100
```

Complete:

Repetición	Nota	Acumulador
1	70	70
2	80	_____
3	90	_____
4	100	_____

Luego:

$$promedio = \frac{suma}{N}$$

Primera solución

Resuelva utilizando `while`.

Reto adicional

Además del promedio, muestre cuántos estudiantes aprobaron con nota mayor o igual a 61.

i ¿Qué estamos integrando?

Variables + acumuladores + contadores + decisiones + ciclos.

Ejercicio 5. Construyamos un menú

El requerimiento

Crear una calculadora que muestre repetidamente el siguiente menú:

```
=====
      CALCULADORA
=====
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Salir
=====
Seleccione una opción:
```

El programa debe solicitar dos números y realizar la operación seleccionada.

Después de mostrar el resultado, debe regresar al menú.

El programa termina únicamente cuando el usuario selecciona la opción 5.

Aquí aparecen dos problemas diferentes

Problema A: seleccionar una operación.

Problema B: repetir el menú hasta que el usuario decida salir.

Antes de programar

¿Qué estructura utilizaría para cada problema?

Necesidad	Estructura propuesta
Seleccionar entre opciones exactas	_____
Repetir el menú	_____
Evitar división entre cero	_____

Diseño general

```
Inicio
Repetir
    Mostrar menú
    Leer opción

    Según opción
        1 -> sumar
        2 -> restar
        3 -> multiplicar
        4 -> dividir
        5 -> salir
        otro -> opción inválida

Mientras opción sea diferente de 5
Fin
```

Pregunta importante

¿Qué debe ocurrir si el usuario selecciona división e ingresa 0 como segundo número?

Ese pequeño detalle convierte un programa que “funciona” en un programa mejor diseñado.

Reto avanzado

Agregue:

6. Potencia

sin alterar el funcionamiento de las opciones anteriores.

Ejercicio 6. Usemos for cuando conocemos las repeticiones

El requerimiento

Una empresa desea registrar las ventas realizadas durante **7 días**.

El programa debe solicitar la venta de cada día y al finalizar mostrar:

- venta total de la semana;
- promedio diario;
- día con mayor venta;
- cantidad de días cuya venta fue mayor a Q1,000.

Pensemos antes de programar

Sabemos exactamente cuántas veces debemos repetir:

7 días

Por lo tanto, podemos expresar naturalmente:

```
Para día = 1 hasta 7
...
Fin Para
```

Identifique las variables necesarias

Complete:

Propósito	Variable posible
Controlar el día	dia
Guardar venta actual	_____
Acumular ventas	_____
Guardar venta mayor	_____
Guardar día de mayor venta	_____
Contar ventas mayores a Q1,000	_____

Pregunta de razonamiento

¿Cómo podemos determinar la venta mayor sin asignar arbitrariamente un número muy grande o muy pequeño?

Pista: piense qué ocurre con **el primer dato ingresado**.

Reto

Resuelva el problema utilizando **for**.

Reto avanzado

Muestre además:

- cuántos días estuvieron por debajo del promedio.

Piense con cuidado: ¿podemos saber esto durante la primera lectura de datos o necesitaríamos conservar información adicional?

! Alto pedagógico

Esta última pregunta es importante.

A veces comprendemos perfectamente un problema, pero descubrimos que la información disponible en ese momento **no es suficiente** para resolver una parte.

Esta necesidad nos prepara para trabajar posteriormente con **arreglos y vectores**.

Ejercicio 7. Dividamos el problema: procedimientos

Hasta ahora hemos colocado prácticamente todo dentro de `main()`.

Eso funciona para programas pequeños. Pero cuando un programa comienza a crecer, necesitamos organizar mejor nuestras soluciones.

El requerimiento

Retomemos la calculadora del ejercicio 5.

En lugar de realizar todas las operaciones dentro de `main()`, queremos separar responsabilidades.

Por ejemplo:

```
void mostrarMenu() {  
    cout << "1. Sumar" << endl;  
    cout << "2. Restar" << endl;  
    cout << "3. Multiplicar" << endl;  
    cout << "4. Dividir" << endl;  
    cout << "5. Salir" << endl;  
}
```

La pregunta cambia

Antes preguntábamos:

¿Qué instrucciones necesita mi programa?

Ahora preguntaremos:

¿Qué tareas diferentes realiza mi programa?

Podemos identificar:

```
mostrarMenu()  
sumar()  
restar()  
multiplicar()  
dividir()
```

Analicemos responsabilidades

Tarea	¿Qué hace?
mostrarMenu()	Presenta las opciones
sumar()	Realiza una suma
restar()	Realiza una resta
multiplicar()	Realiza una multiplicación
dividir()	Realiza una división

Primer objetivo

Construya una versión donde cada operación se encuentre en un procedimiento independiente.

Ejemplo:

```
void sumar() {
    double a, b;

    cout << "Ingrese el primer número: ";
    cin >> a;

    cout << "Ingrese el segundo número: ";
    cin >> b;

    cout << "Resultado: " << a + b << endl;
}
```

Luego main() solamente coordina:

```
int main() {

    int opcion;

    do {
        mostrarMenu();
        cin >> opcion;

        switch (opcion) {
            case 1:
                sumar();
        }
    } while (opcion > 0);
}
```

```

        break;

        // continuar...

    }

} while (opcion != 5);

return 0;
}

```

Detengámonos aquí

Observe algo importante.

`main()` comienza a ser más fácil de leer porque ya no contiene todos los detalles.

Podemos leerlo casi como una historia:

```

mostrar menú
leer opción
ejecutar operación
repetir

```

💡 Idea fundamental

Un procedimiento nos permite **darle nombre a una tarea**.

En lugar de pensar en veinte instrucciones, podemos pensar:

“Ahora quiero sumar.”

y delegar esa responsabilidad a `sumar()`.

Preguntas de comprensión

1. ¿Qué ventaja tiene separar el programa?
2. ¿Qué ocurre si necesitamos sumar desde dos lugares diferentes?
3. ¿Estamos repitiendo código?
4. ¿Quién controla el flujo general del programa?
5. ¿Qué responsabilidad debería tener `main()`?

Ejercicio 8. Funciones y parámetros: piezas reutilizables

Este es el punto en el que queremos poner especial atención.

El problema de nuestra solución anterior

Observe este procedimiento:

```
void sumar() {  
    double a, b;  
  
    cout << "Ingrese el primer número: ";  
    cin >> a;  
  
    cout << "Ingrese el segundo número: ";  
    cin >> b;  
  
    cout << "Resultado: " << a + b << endl;  
}
```

Funciona.

Pero `sumar()` está haciendo demasiadas cosas:

1. solicita datos;
2. realiza el cálculo;
3. muestra el resultado.

¿Qué ocurriría si los números ya estuvieran disponibles?

¿Qué ocurriría si quisiéramos utilizar la suma sin pedir nuevamente los datos?

Aquí aparecen los **parámetros**.

8.1 De procedimiento cerrado a procedimiento reutilizable

Podemos escribir:

```
void sumar(double a, double b) {  
    cout << "Resultado: " << a + b << endl;  
}
```

Y utilizarlo así:

```
sumar(10, 20);
```

o:

```
double x = 5;  
double y = 8;  
  
sumar(x, y);
```

¿Qué cambió?

`sumar()` ya no necesita saber de dónde vienen los números.

Solamente recibe lo que necesita para trabajar.

i Parámetros

Los parámetros permiten enviar información a un procedimiento o función.
Podemos imaginarlo así:

```
main()  
|  
|  a = 10  
|  b = 20  
v  
sumar(a, b)
```

El procedimiento recibe los datos y realiza su tarea.

8.2 Ahora necesitamos devolver un resultado

Supongamos que queremos guardar la suma:

```
resultado = sumar(a, b);
```

Entonces ya no queremos solamente mostrar el resultado.

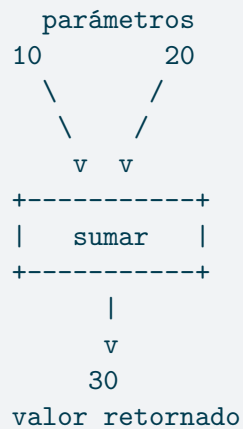
Queremos que la función **devuelva** un valor.

```
double sumar(double a, double b) {  
    return a + b;  
}
```

Ahora podemos hacer:

```
double resultado;  
  
resultado = sumar(10, 20);  
  
cout << resultado << endl;
```

8.3 Pensemos en la función como una pequeña máquina



Una función puede verse como un pequeño algoritmo:

ENTRADAS → PROCESO → SALIDA

Exactamente la misma lógica con la que comenzamos el curso.

8.4 El reto integrador

Construya nuevamente la calculadora utilizando funciones.

Debe incluir:

```
double sumar(double a, double b);
double restar(double a, double b);
double multiplicar(double a, double b);
double dividir(double a, double b);
void mostrarMenu();
```

Organización propuesta

main() será responsable de:

1. mostrar el menú;
2. leer la opción;
3. solicitar los números;
4. llamar a la función correspondiente;
5. recibir el resultado;
6. mostrarlo;
7. repetir hasta seleccionar salir.

Las funciones serán responsables **únicamente de realizar su operación**.

Estructura inicial

```
#include <iostream>
using namespace std;

void mostrarMenu();
double sumar(double a, double b);
double restar(double a, double b);
double multiplicar(double a, double b);
double dividir(double a, double b);

int main() {

    // Diseñe aquí el programa principal.
```

```

    return 0;
}

void mostrarMenu() {
    // Mostrar opciones.
}

double sumar(double a, double b) {
    return a + b;
}

double restar(double a, double b) {
    // completar
}

double multiplicar(double a, double b) {
    // completar
}

double dividir(double a, double b) {
    // completar
}

```

⚠ Una función no elimina el análisis

Crear funciones no significa fragmentar el código porque sí.

Cada función debe tener una responsabilidad clara.

Pregúntese:

¿Qué recibe? → ¿Qué hace? → ¿Qué devuelve?

8.5 Segundo reto: función para determinar el mayor

Construya:

```
int mayor(int a, int b);
```

Debe recibir dos números enteros y devolver el mayor.

Ejemplo:

```
int resultado = mayor(15, 8);
```

Resultado:

```
15
```

Preguntas

- ¿Cuáles son los parámetros?
 - ¿Qué tipo de dato devuelve?
 - ¿Necesita `cout` dentro de la función?
 - ¿Necesita `cin` dentro de la función?
 - ¿Dónde debería mostrarse el resultado?
-

8.6 Tercer reto: función para determinar si un número es par

Diseño:

```
bool esPar(int numero);
```

Ejemplos:

```
esPar(8)  -> true  
esPar(7)  -> false
```

Pista:

```
numero % 2
```

Pregunta importante

¿Por qué `bool` parece un tipo de retorno apropiado para esta función?

8.7 Cuarto reto: combine funciones

Construya un programa que solicite **N números**.

Para cada número debe utilizar:

```
bool esPar(int numero);
```

Al finalizar debe mostrar:

- cantidad de números pares;
- cantidad de números impares.

Aquí estamos integrando:

```
variables
+
ciclo
+
función
+
parámetros
+
decisión
```

Antes de escribir código

Dibuje mentalmente el flujo:

```
main()
|
| lee número
|
+----> esPar(numero)
|
|         +----> true / false
|
| actualiza contador
|
+----> siguiente número
```

! Este es el objetivo de la clase

No memorizar una función.

Comprender que un programa complejo puede construirse combinando **pequeñas soluciones que ya sabemos resolver**.

Desafío final. Sistema de evaluación de hábitos e IMC

Situación

Una clínica universitaria desea contar con un programa sencillo que permita realizar una **evaluación educativa básica** de algunos hábitos de alimentación y actividad física de una persona y calcular su **Índice de Masa Corporal (IMC)**.

El programa no debe limitarse a calcular un número. Debe recopilar información, analizarla, tomar decisiones y presentar un reporte final comprensible.

! Importante

Este ejercicio tiene fines académicos para practicar análisis y programación.

El IMC es un indicador de referencia y **no constituye por sí solo un diagnóstico médico**. Las recomendaciones del programa deben redactarse como orientaciones generales de hábitos saludables, no como tratamientos médicos personalizados.

El requerimiento

El sistema deberá solicitar inicialmente:

- nombre;
- edad;
- sexo;
- peso en kilogramos;
- estatura en metros.

Posteriormente deberá preguntar por algunos hábitos, por ejemplo:

¿Cuántos días por semana consume comida rápida (pizza, hamburguesas, pollo frito, etc.)?

¿Cuántos días por semana consume golosinas, frituras, pastelitos u otras chucherías?

¿Cuántos días por semana consume bebidas azucaradas?

¿Cuántas bebidas energizantes consume por semana?

¿Cuántos días por semana consume bebidas alcohólicas?

¿Cuántos días por semana consume frutas o verduras?

¿Cuántos días por semana realiza al menos 30 minutos de actividad física?

Los valores deberán ser validados cuando corresponda. Una frecuencia expresada en días por semana, por ejemplo, no debería aceptar valores como -2, 9 o 15.

Cálculo del IMC

El programa deberá calcular:

$$IMC = \frac{peso}{estatura^2}$$

Para **personas adultas**, utilice la siguiente clasificación didáctica:

IMC	Clasificación
Menor de 18.5	Bajo peso
18.5 a 24.9	Peso saludable
25.0 a 29.9	Sobrepeso
30.0 o más	Obesidad

⚠ Validación necesaria

Como esta clasificación corresponde a adultos, el programa debe verificar la edad. Si la persona es menor de 18 años, deberá indicar que esta clasificación no será aplicada y evitar emitir una clasificación de adulto.

Pero el IMC no es todo el problema

Una persona podría obtener un IMC dentro del rango saludable y, al mismo tiempo, reportar hábitos poco saludables. Por eso el programa deberá analizar **dos componentes**:

1. Resultado del IMC
2. Hábitos reportados

y construir un reporte final utilizando ambos.

Recomendaciones que puede generar el programa

Las recomendaciones deben depender de la información ingresada. Algunos ejemplos de reglas son:

```
Si consume comida rápida con mucha frecuencia:  
    recomendar reducir su frecuencia y preferir  
    preparaciones caseras con alimentos variados.  
  
Si consume muchas golosinas, frituras o bebidas azucaradas:  
    recomendar disminuir su consumo y preferir  
    agua y alimentos menos procesados.  
  
Si consume bebidas energizantes:  
    recomendar limitar o evitar su consumo habitual.  
  
Si reporta consumo frecuente de alcohol:  
    mostrar una recomendación preventiva para reducirlo  
    o evitarlo.  
  
Si consume pocas frutas o verduras:  
    recomendar aumentar su presencia en las comidas.  
  
Si realiza poca actividad física:  
    recomendar incorporar actividad física regularmente,  
    de acuerdo con sus posibilidades y condición personal.
```

Estas reglas son una orientación. **El estudiante deberá proponer y justificar su propia lógica.**

Ejemplo del reporte esperado

```
=====
REPORTE DE EVALUACIÓN
=====

Nombre: Ana López
Edad: 22 años
Peso: 68 kg
Estatura: 1.65 m

IMC: 24.98
Clasificación: Peso saludable

-----
HÁBITOS REPORTADOS
-----

Comida rápida: 4 días/semana
Chucherías: 5 días/semana
Bebidas azucaradas: 3 días/semana
Bebidas energizantes: 2 por semana
Alcohol: 1 día/semana
Frutas o verduras: 2 días/semana
Actividad física: 1 día/semana

-----
RECOMENDACIONES GENERALES
-----

- Su IMC se encuentra en el rango de referencia
  considerado saludable para adultos.
- Considere disminuir la frecuencia de comida rápida.
- Reduzca golosinas, frituras y bebidas azucaradas.
- Prefiera agua como bebida habitual.
- Aumente gradualmente el consumo de frutas y verduras.
- Procure incorporar actividad física con regularidad.

=====
```

Observe que el programa **no imprime siempre el mismo mensaje**. Las recomendaciones deben construirse según las respuestas del usuario.

Antes de programar: analice

No se calificará una solución que comience directamente con el código.

A. Comprensión del requerimiento

Explique con sus propias palabras qué problema debe resolver el programa.

B. Entradas

Identifique todos los datos que necesita solicitar y construya una tabla con variable, tipo y propósito.

C. Validaciones

Determine qué datos necesitan ser validados. Pregúntese:

- ¿puede existir una estatura igual a cero?;
- ¿puede existir un peso negativo?;
- ¿pueden existir 10 días en una semana?;
- ¿qué debe hacer el programa si el usuario ingresa un dato inválido?

D. Procesos

Como mínimo deberá considerar:

```
calcular IMC  
clasificar IMC  
analizar hábitos  
generar recomendaciones
```

E. Decisiones

Identifique las decisiones que deberá tomar el algoritmo:

```
¿La persona es adulta?  
¿En qué rango se encuentra el IMC?  
¿Consume comida rápida frecuentemente?  
¿Consume bebidas energizantes?  
¿Consume suficientes frutas o verduras?  
¿Realiza actividad física regularmente?
```

F. Repeticiones

Analice si necesita ciclos para validar datos, repetir preguntas, mostrar un menú o permitir evaluar a otra persona.

G. Descomposición del problema

Divida el programa en pequeñas responsabilidades. Antes de escribir una función, pregúntese:

¿Qué recibe? → ¿Qué hace? → ¿Qué devuelve?

Podrían existir funciones como:

```
double calcularIMC(double peso, double estatura);  
string clasificarIMC(double imc);  
bool frecuenciaAlta(int dias);  
void mostrarRecomendaciones(...);
```

Estas firmas son únicamente ejemplos. El estudiante puede diseñar una solución diferente si puede **explicar y defender su decisión**.

Requisitos mínimos de programación

La solución deberá integrar de manera coherente:

- variables y tipos de datos;
- entrada y salida;
- operadores aritméticos;
- operadores relacionales y lógicos;
- bifurcaciones;
- ciclos;
- validación de datos;

- al menos un menú o mecanismo para repetir la evaluación;
- procedimientos;
- funciones;
- parámetros;
- valores retornados;
- casos de prueba.

💡 El objetivo no es hacer el programa más largo

El objetivo es demostrar que puede tomar un requerimiento, analizarlo, dividirlo y construir una solución utilizando las herramientas aprendidas durante el curso.

Entregables

El estudiante deberá entregar:

1. **Análisis del requerimiento.**
2. **Entradas, procesos y salidas.**
3. **Listado de variables y tipos de datos.**
4. **Diagrama de flujo.**
5. **Pseudocódigo.**
6. **Código fuente en C++.**
7. **Funciones utilizadas y explicación de sus parámetros.**
8. **Al menos cinco casos de prueba.**
9. **Captura o evidencia de ejecución.**
10. **Breve conclusión:** qué parte del problema le resultó más difícil y cómo la resolvió.

❗ No se calificará únicamente el código

El propósito del desafío es evaluar el proceso completo:

comprender → analizar → diseñar → programar → probar → explicar

Un programa que funcione pero que el estudiante no pueda explicar no demuestra completamente el aprendizaje esperado.

Preparación para la exposición

El lunes se seleccionarán estudiantes al azar para explicar su solución.

Deberán estar preparados para responder preguntas como:

¿Por qué utilizó esa variable?
 ¿Por qué utilizó un if en esa parte?
 ¿Qué ocurriría si cambia esta condición?
 ¿Por qué eligió while, do-while o for?
 ¿Qué parámetros recibe esta función?
 ¿Qué devuelve?
 ¿Qué ocurriría si eliminamos esta función?
 ¿Cómo validó los datos?
 ¿Qué caso de prueba utilizó para un valor límite?
 ¿Cómo agregaría una nueva recomendación?

⚠ Importante

No se trata de memorizar el código para la exposición.
 Debe ser capaz de **explicar la lógica de su propia solución**.

¿Qué hemos recuperado?

Durante estos ocho ejercicios recorrimos nuevamente los principales elementos del curso, pero no como temas aislados.

Los utilizamos como **herramientas para resolver problemas**.

Nivel	Herramienta	Pregunta que resuelve
1	Variables	¿Qué información necesito guardar?
2	if	¿Debo tomar una decisión?
3	if / else if	¿Existen varios caminos?
4	while	¿Debo repetir mientras se cumpla una condición?
5	Menú + switch	¿Debo seleccionar una acción?
6	for	¿Sé cuántas veces debo repetir?
7	Procedimientos	¿Puedo separar una tarea?
8	Funciones + parámetros	¿Puedo crear una solución reutilizable que reciba y devuelva información?

La idea que debe quedar completa

Observe cómo evolucionó nuestro pensamiento:

```
Tengo un problema
  ↓
¿Qué me están pidiendo?
  ↓
¿Qué sé?
  ↓
¿Qué necesito?
  ↓
¿Qué debo calcular o decidir?
  ↓
¿Necesito repetir?
  ↓
¿Puedo dividir el problema?
  ↓
¿Qué funciones necesito?
  ↓
Escribo el algoritmo
  ↓
Lo convierto en C++
  ↓
Lo pruebo
```

Consejo final del Ingeniero

No intente recordar “qué código llevaba aquel ejercicio”.

Pregúntese:

¿Qué necesita hacer este problema?

Cuando comprende la necesidad, puede elegir la herramienta adecuada.

La programación deja de ser una colección de instrucciones y comienza a convertirse en una forma de resolver problemas.

Semáforo de aprendizaje

Antes de continuar, marque honestamente su situación actual.

Competencia	Necesito ayuda	Puedo hacerlo con orientación	Puedo hacerlo solo
Identifico entradas y salidas			
Selecciono variables apropiadas			
Construyo operaciones			
Utilizo bifurcaciones			
Construyo ciclos			
Construyo un menú			
Utilizo <code>for</code>			
Comprendo procedimientos			
Comprendo parámetros			
Construyo funciones			
Puedo pasar del problema al código			

El objetivo no es que todos marquen verde.

El objetivo es identificar **qué pieza falta** para poder completarla.

Próximo paso: arreglos y vectores

En el ejercicio de las ventas semanales apareció una dificultad:

¿Qué hacemos si necesitamos conservar varios valores para utilizarlos después?

Hasta ahora hemos trabajado principalmente con variables individuales:

```
double nota;  
double venta;  
int numero;
```

Pero pronto necesitaremos guardar conjuntos de datos:

```
nota 1  
nota 2  
nota 3  
...  
nota N
```

Ahí aparece nuestra siguiente herramienta:

arreglos y vectores.

Ya no llegaremos a ellos como un tema aislado.

Llegaremos porque ahora existe un problema que nuestras variables individuales ya no resuelven cómodamente.

! Puente hacia la siguiente clase

Primero aprendimos a guardar **un dato**.

Ahora aprenderemos a guardar y procesar **muchos datos relacionados**.

Ese será nuestro siguiente paso.